Laboratorio 5 — Hoja de trabajo

El catálogo con objetos (cierre de la Actividad 1) · Tecnología Web II

Nombre: Ronald Salvatierra Flores **Fecha:** 14 / 10 / 2026

Comandos usados: php demo.php · php verificar.php (14 pruebas) · php -S 127.0.0.1:8000 -t public (ejecutados dentro de Docker con docker compose exec php).

Parte A — Antes de tocar nada

Salida de php demo.php antes de completar los TODO (se ejecutó el código original):

```
1) Un objeto:            Miel multifloral 500 g — Bs 45,00
2) Método con reglas:     precio de feria Bs 40,50
3) Después de vender 30:  stock 2 (bajo)
7) $otra = $p; reponer:   $p->stock() = 1010  ($otra === $p: true)
8) clone:                 $p->stock() = 1010, $copia->stock() = 1110
9) Catálogo:              6 productos, inventario Bs 0,00
10) Más barato y más caro: Miel multifloral 500 g / Miel multifloral 500 g
```

1. ¿Qué pasó en el paso 4 (vender 5 cuando quedaban 2)? ¿Cuánto stock quedó realmente?

No apareció ningún error ni mensaje: la línea 4 ni siquiera se imprimió, porque no se lanzó ninguna excepción. La venta se aceptó y el stock quedó en −3 (var_dump($p->stock()) mostró int(-3)). Después de reponer 10, el stock mostró 1010 en el paso 7 porque en el paso 5 se logró asignar $p->stock = 1000.

2. ¿Qué pasó en los pasos 5 y 6? ¿Apareció algún mensaje?

No apareció ningún mensaje. En el paso 5 la asignación $p->stock = 1000 funcionó desde afuera porque la propiedad era pública, y se saltó todas las reglas. En el paso 6 se creó sin queja un producto con código "miel" y precio −5, que son datos inválidos.

3. ¿Cuánto vale el inventario (paso 9)? ¿Es correcto?

Mostró Bs 0,00. No es correcto: el valor real es Bs 5.030,00, el mismo del laboratorio 2. Ocurría porque valorInventario() todavía devolvía 0.0 (TODO 6). Además, ordenadosPorPrecio() no ordenaba nada (TODO 5): el más barato y el más caro salían iguales.

4. En tus palabras: ¿qué problemas tendría la asociación si el sistema funciona así?

Tendría datos falsos: stock negativo, precios o códigos inválidos y productos duplicados que reemplazan a otros sin avisar. El inventario saldría en cero, por lo que no se podrían tomar buenas decisiones de compra o venta. Y como cualquier parte del programa puede cambiar el stock, no habría forma de saber dónde se rompió una regla.

Parte B — Un objeto que se defiende solo (TODO 1, 2 y 3)

1. Salida de las líneas 4, 5 y 6 después de completar los TODO

```
4) Vender 5 más:          Stock insuficiente de MIE-500: hay 2, se pidieron 5.
5) $p->stock = 1000:      Cannot access private property Producto::$stock
6) Objeto inválido:       Código inválido: miel
```

2. ¿Por qué validar en el constructor y no en cada página que crea productos?

Porque el constructor es el único punto por el que pasa todo producto al crearse. Si la regla está ahí, es imposible que exista un Producto inválido, sin importar qué página o script lo cree. Si se validara en cada página habría que repetir el código y bastaría olvidarlo en una para dejar entrar datos malos.

3. ¿Qué diferencia hay entre private, private readonly y public? Un ejemplo de cada uno en Producto.

public: se puede leer y cambiar desde cualquier lugar. Antes lo era $stock, y por eso se pudo hacer $p->stock = 1000. Hoy son públicos los métodos como codigo() o vender().

private: solo se accede desde dentro de la clase. Ejemplo: private int $stock y private float $precio; el precio solo cambia con cambiarPrecio().

private readonly: es privado y además solo se asigna una vez, en el constructor. Ejemplo: private readonly string $codigo; el código de un producto nunca cambia.

4. ¿Por qué se usan dos excepciones distintas (InvalidArgumentException y DomainException)?

InvalidArgumentException indica que el dato recibido es inválido por sí mismo, sin importar el estado del objeto (código mal escrito, precio ≤ 0, cantidad ≤ 0). DomainException indica que el dato es válido, pero rompe una regla del negocio en ese momento: por ejemplo, vender más de lo que hay en stock o agregar un código que ya existe. Con tipos distintos, el programa puede capturar y tratar cada caso por separado.

Parte C — El catálogo (TODO 4, 5 y 6)

1. Antes del TODO 4, ejecutar el comando de la hoja. ¿Qué pasó con la miel original?

```
Resultado: 6 Otra miel
```

La miel original desapareció. agregar() asignó el nuevo producto a la misma clave "MIE-500" y reemplazó al anterior sin avisar. La cantidad siguió en 6 y buscar("MIE-500") devolvió "Otra miel". Se perdió un producto y nadie se enteró; por eso el TODO 4 lanza DomainException.

2. ¿Cuál es el valor del inventario? ¿Coincide con el laboratorio 2?

Bs 5.030,00. Sí coincide con el del laboratorio 2; el verificador lo confirma con valorInventario() = 5030.

3. ¿Qué devuelve $a <=> $b en cada caso?

Si $a es menor que $b devuelve un valor negativo (−1). Si son iguales devuelve 0. Si $a es mayor que $b devuelve un valor positivo (1). usort usa ese resultado para decidir el orden; al invertir los operandos ($b <=> $a) el orden queda descendente.

4. ¿Por qué desdeArchivo es un método static?

Porque se usa para crear el catálogo, cuando todavía no existe ningún objeto Catalogo sobre el cual llamar un método. Al ser static se llama con la clase (Catalogo::desdeArchivo(...)) y devuelve un objeto nuevo ya cargado. Es un método "fábrica".

Parte D — Objetos y referencias (pasos 7 y 8 de la demo)

1. Después de $otra = $p; $otra->reponer(10);, ¿por qué cambió también $p?

Porque en PHP una variable de objeto guarda un identificador que apunta al objeto, no una copia. $otra y $p apuntan al mismo objeto, así que un cambio hecho con una se ve en la otra. La demo lo confirma: $otra === $p dio true y el stock fue 12 en ambas.

2. ¿Qué hace clone? ¿Cuándo lo usarías?

clone crea un objeto nuevo e independiente con los mismos valores. En el paso 8, $copia->reponer(100) subió $copia a 112 y $p siguió en 12. Lo usaría cuando necesite trabajar con una copia sin modificar el original, por ejemplo para simular una venta o probar un cambio de precio.

Parte E — Verificación

php verificar.php: 14 de 14 comprobaciones. Salida real de la ejecución:

```
PS C:\Users\RONALD\Documents\TW2\Lab05\inicio>    Set-Location "$TW2\Lab05\inicio"
>>    docker compose down
>>    docker compose up -d
>>    docker compose exec php php -l src/Producto.php
>>    docker compose exec php php -l src/Catalogo.php
>>    docker compose exec -T php php verificar.php
>>    docker compose exec php php demo.php
>>    Start-Process 'http://127.0.0.1:8000/'
[+] down 2/2
 ⠿ Container inicio-php-1 Removed
 ⠿ Network inicio_default Removed
[+] up 2/2
 ⠿ Network inicio_default Created
 ⠿ Container inicio-php-1 Started
No syntax errors detected in src/Producto.php
No syntax errors detected in src/Catalogo.php
OK    Producto válido: getters y valor en stock (45 × 32 = 1440)
OK    El constructor rechaza código, precio o stock inválidos (InvalidArgumentException)
OK    vender() descuenta y no permite vender más de lo que hay (DomainException)
OK    Encapsulamiento: $p->stock = 1000 no está permitido
OK    estadoStock: 0 → agotado, 5 → bajo, 6 → normal
OK    precioConDescuento(10) de 85 → 76.5 y rechaza 60 %
OK    Catalogo::desdeArchivo carga 6 productos en 3 categorías
OK    valorInventario() = 5030 (el mismo resultado que el lab 2)
OK    buscar() normaliza el código y devuelve null si no existe
OK    agregar() rechaza un código repetido (DomainException)
OK    ordenadosPorPrecio(): propóleo 38 primero; descendente: cera 120 primero
OK    deCategoria(Derivados) y disponibles() (5 de 6: la miel de cusi está agotada)
OK    La página agrupa por categoría, muestra el inventario y escapa "Polen &amp;amp; propóle
OK    La ficha muestra el precio de feria (Bs 76,50) y un código inexistente da 404

14 de 14 comprobaciones
1) Un objeto:            Miel multifloral 500 g — Bs 45,00
2) Método con reglas:     precio de feria Bs 40,50
3) Después de vender 30:  stock 2 (bajo)
4) Vender 5 más:          Stock insuficiente de MIE-500: hay 2, se pidieron 5.
5) $p->stock = 1000:      Cannot access private property Producto::$stock
6) Objeto inválido:       Código inválido: miel
7) $otra = $p; reponer:   $p->stock() = 12  ($otra === $p: true)
8) clone:                 $p->stock() = 12, $copia->stock() = 112
9) Catálogo:              6 productos, inventario Bs 5.030,00
10) Más barato y más caro: Propóleo en gotas 30 ml / Cera pura 1 kg
PS C:\Users\RONALD\Documents\TW2\Lab05\inicio>
```

Figura 1. Verificador (14 de 14) y demo.php después de completar los TODO.

Páginas en el navegador (http://localhost:8000):



Figura 2. Página principal: 6 productos, 5 disponibles, inventario Bs 5.030,00.

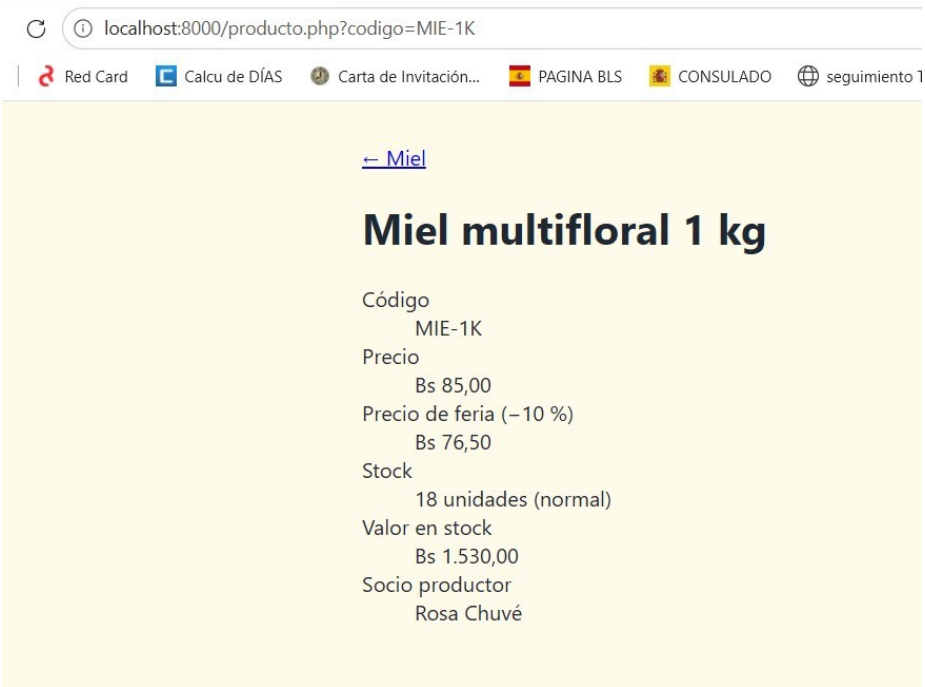


Figura 3. Ficha de Miel multifloral 1 kg: precio de feria Bs 76,50.

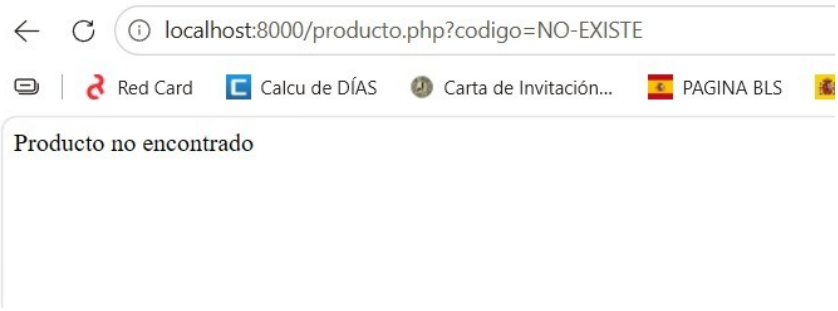


Figura 4. Código inexistente (NO-EXISTE): "Producto no encontrado" (404).

Parte F — Entrega integrada de la Actividad 1 (labs 1 a 5)

Lab	Concepto principal	¿Dónde aparece en el lab 5?
1	Fundamentos de PHP: variables, tipos de datos y salida de texto.	Los tipos de las propiedades (string, int, float) y los textos que imprime demo.php.
2	Funciones y arreglos; cálculo del inventario con valor_inventario().	Catalogo::valorInventario() da el mismo resultado: Bs 5.030,00.
3	Ordenar, filtrar y recorrer colecciones de datos con funciones.	ordenadosPorPrecio(), deCategoria() y disponibles() dentro de Catalogo.
4	Páginas web con PHP: mostrar el catálogo desde un archivo JSON y escapar el HTML.	public/index.php y producto.php leen datos/productos.json; e() escapa "Polen & propóleo".
5	Programación orientada a objetos: clases, constructor, encapsulamiento y excepciones.	Producto, Categoria y Catalogo agrupan datos y reglas; el verificador da 14 de 14.

Reflexión

Compara valor_inventario() del laboratorio 2 con Catalogo::valorInventario(). ¿Qué ganaste al pasar a objetos?

El resultado es el mismo (Bs 5.030,00), pero cambió dónde viven los datos y las reglas. Antes una función suelta recorría arreglos y cualquiera podía modificar los datos. Ahora cada Producto sabe calcular su valorEnStock() y valida lo que recibe, y el Catalogo solo suma esos valores. Gané datos que no pueden quedar inválidos, errores claros con excepciones, código más fácil de reutilizar y de probar, y una verificación automática (14 de 14) que avisa si algo se rompe.

Qué cambié y qué aprendí

Producto.php: validé código, precio y stock en el constructor; agregué la regla de stock suficiente en vender(); y cambié $stock a private para que solo cambie con vender() y reponer().

Catalogo.php: rechacé códigos duplicados en agregar(), ordené con usort y <=> en ordenadosPorPrecio(), y calculé el inventario con array_map y array_sum.

Aprendí que un objeto debe defenderse solo (encapsulamiento y excepciones), que los objetos se pasan por referencia, y que las pruebas automáticas complementan la revisión manual.

Nota técnica: el verificador necesitaba un archivo Verificador.php que no venía en el material del laboratorio; se agregó uno mínimo que solo ejecuta las pruebas y cuenta los resultados, sin modificar verificar.php.